

Python Loops Guide

A Beginner-Friendly Guide to Repeating Tasks

Learning Objectives

By the end of this guide, you will be able to:

- Explain why loops are useful.
- Use for loops to repeat code.
- Use while loops when a condition controls repetition.
- Work with range() and sequences.
- Use break, continue, and pass.
- Recognize and avoid common loop mistakes.

1. What Is a Loop?

A loop repeats a block of code. Loops are useful when you need to perform the same task many times, such as displaying a series of numbers or processing items in a list. Python has two main loop statements: for and while.

2. The for Loop

A for loop goes through the items in a sequence, such as a list, string, or range of numbers. The indented statements run once for each item.

Syntax:

```
for item in sequence:  
    # code to repeat
```

Example:

```
for number in [1, 2, 3, 4, 5]:  
    print(number)
```

3. Using range()

The range() function produces a sequence of integers, commonly used with for loops. The ending value is not included.

Examples:

```
for i in range(5):  
    print(i)  
  
# Output: 0, 1, 2, 3, 4
```

You can provide a start and stop value:

```
for i in range(1, 6):  
    print(i)  
  
# Output: 1, 2, 3, 4, 5
```

You can also provide a step:

```
for i in range(2, 11, 2):  
    print(i)  
  
# Output: 2, 4, 6, 8, 10
```

4. Looping Through a String

A string contains characters, and a for loop can visit each character in order.

```
for letter in "Python":  
    print(letter)
```

5. Looping Through a List

A list can hold multiple values. A for loop can process each value in the list.

```
fruits = ["apple", "banana", "mango"]  
  
for fruit in fruits:  
    print(fruit)
```

6. The while Loop

A while loop repeats as long as its condition remains True. Make sure something in the loop can eventually make the condition False, or the loop may never end.

Example:

```
count = 1  
  
while count <= 5:  
    print(count)  
    count += 1
```

The statement `count += 1` increases count by one each time. When count becomes 6, the condition is False and the loop stops.

7. for Loop vs while Loop

for is commonly used when going through a sequence or a known range of items. **while** is commonly used when repetition depends on a condition. Either can be appropriate depending on the task.

8. The break Statement

`break` stops the nearest enclosing loop immediately.

```
for number in range(1, 10):  
    if number == 5:  
        break
```

```
print(number)
```

This prints 1, 2, 3, and 4. When number is 5, the loop ends.

9. The continue Statement

continue skips the rest of the current loop iteration and moves to the next iteration.

```
for number in range(1, 6):  
    if number == 3:  
        continue  
    print(number)
```

This prints 1, 2, 4, and 5; the number 3 is skipped.

10. The pass Statement

pass does nothing. It can be used as a placeholder when you plan to add code later.

```
for number in range(3):  
    pass
```

11. Nested Loops

A nested loop is a loop inside another loop. For each iteration of the outer loop, the inner loop runs through its own iterations.

Example:

```
for row in range(1, 4):  
    for column in range(1, 3):  
        print(row, column)
```

12. Using a Loop to Calculate a Total

Loops can add values together or perform other repeated calculations.

```
numbers = [2, 4, 6, 8]  
total = 0  
  
for number in numbers:  
    total += number  
  
print("Total:", total)
```

13. Looping with an if Statement

You can combine loops and conditions to filter or classify values.

```
for number in range(1, 11):  
    if number % 2 == 0:  
        print(number, "is even")
```

14. Common Loop Mistakes

- Forgetting the colon at the end of for or while statements.
- Incorrect indentation inside the loop.
- Creating an infinite while loop by never changing its condition.

- Expecting range(5) to include 5; it stops before 5.
- Using break or continue without understanding which loop they affect.

15. Practice Exercises

- 1 Print numbers from 1 to 10 using a for loop.
- 2 Print even numbers from 2 to 20.
- 3 Use a while loop to count down from 5 to 1.
- 4 Calculate the sum of numbers from 1 to 100.
- 5 Print every character in your name.
- 6 Use continue to skip multiples of 3 from 1 to 15.
- 7 Use nested loops to print a small rectangle of stars.

16. Mini Project — Number Table

Write a program that displays the multiplication table for a number entered by the user.

Example solution:

```
number = int(input("Enter a number: "))

for i in range(1, 11):
    print(number, "x", i, "=", number * i)
```

17. Chapter Summary

Loops repeat code. A for loop iterates over a sequence, while a while loop repeats as long as its condition is True. range() generates integer sequences. break exits a loop, continue skips to the next iteration, and pass acts as a placeholder. Indentation and careful condition updates help loops work correctly.

Quick Review

- Use for to iterate through a sequence.
- Use while to repeat while a condition is True.
- range(stop) excludes the stop value.
- break exits a loop.
- continue skips the current iteration.
- pass is a placeholder statement.